



Do Docker ao Kubernetes — TP2

Nome: Guilherme de Moraes China Costa

Data: 06/09/2026

Planejamento da aplicação

1. Aplicação proposta

O projeto chama-se **Eventos Microservices** e permite consultar itens disponíveis e criar pedidos. A proposta é separar bem as duas partes da aplicação e acompanhar o caminho de publicação: primeiro localmente, depois em Docker e por fim no Kubernetes.

Microserviço	Responsabilidade	Endpoints principais
product-service	Mantém o catálogo de eventos disponíveis. Para o escopo do trabalho, os dados permanecem em memória.	GET /products GET /products/{id} POST /products
order-service	Cria e consulta reservas. Antes de criar uma reserva, consulta o catálogo para verificar se o evento existe.	GET /orders GET /orders/{id} POST /orders

Cada serviço roda no seu próprio processo, usa sua própria porta e gera sua própria imagem. O order-service não mexe diretamente nos dados do catálogo: quando precisa saber se um evento existe, faz uma chamada HTTP para o product-service. Isso deixa bem claro onde começa e termina a responsabilidade de cada um.

2. Máquina virtual × container

Característica	Máquina virtual	Container
Sistema operacional	Cada VM executa um sistema operacional convidado completo sobre um hypervisor.	Compartilha o kernel do hospedeiro e inclui aplicação e dependências necessárias.
Consumo de recursos	Maior uso de CPU, memória e disco, pois há um SO por VM.	Menor uso de recursos; no projeto, a imagem final possui somente JRE e JAR.
Inicialização	Depende da inicialização do sistema operacional e costuma levar mais tempo.	Inicia o processo da aplicação diretamente, normalmente em segundos.
Isolamento	Isolamento por virtualização de hardware.	Isolamento por processos, namespaces e cgroups do sistema operacional.

Com máquinas virtuais, seria necessário preparar uma máquina para cada serviço, instalar Java e copiar os JARs. Com containers isso fica bem mais leve: cada serviço vai dentro da sua própria imagem e pode ser executado da mesma forma em qualquer computador com Docker. Isso ajuda bastante em microserviços, porque permite atualizar ou aumentar somente o serviço que precisa de atenção. Por exemplo, uma mudança nas reservas não obriga a publicar o catálogo de novo.

Microsserviços Spring Boot

3. product-service

O product-service foi feito em Java 21 com Spring Boot e sobe na porta 8081. Para não fugir do foco do trabalho, deixei os dados em memória e cadastrei dois eventos iniciais. Ele permite listar todos os eventos, buscar um evento pelo id e cadastrar um novo.

```
@RestController
@RequestMapping("/products")
public class ProductController {
    @GetMapping public List<Product> findAll() { ... }
    @GetMapping("/{id}") public Product findById(@PathVariable Long id) { ... }
    @PostMapping @ResponseStatus(HttpStatus.CREATED)
    public Product create(@RequestBody ProductRequest request) { ... }
}
```

4. order-service

O order-service é outro projeto Spring Boot, separado do primeiro, e roda na porta 8082. Ele guarda as reservas em memória. Antes de aceitar uma reserva, ele pergunta ao catálogo se o evento informado existe. Se não existir, o pedido não é criado e a API retorna um erro adequado.

```
public Order create(OrderRequest request) {
    if (request.productId() == null || !productClient.exists(request.productId())) {
        throw new ProductUnavailableException(request.productId());
    }
    Long id = sequence.incrementAndGet();
    Order order = new Order(id, request.productId(), request.quantity(), "CRIADO");
    orders.put(id, order);
    return order;
}
```

Para gerar e executar ambos os serviços localmente: `mvn clean package`, depois `java -jar product-service/target/product-service-1.0.0.jar` e `java -jar order-service/target/order-service-1.0.0.jar`. O exemplo POST `http://localhost:8082/orders`, com `{"productId":1,"quantity":2}`, cria uma reserva para um evento existente.

Docker e comunicação entre microsserviços

5. Imagens Docker

Há um Dockerfile para cada microsserviço. Os dois usam duas etapas: primeiro o Maven com Java 21 gera o JAR; depois uma imagem menor, só com o JRE, executa a aplicação. Assim, as imagens finais ficam mais enxutas. Os nomes usados são `eventos/product-service:1.0.0` e `eventos/order-service:1.0.0`.

```
docker build -t eventos/product-service:1.0.0 ./product-service
docker build -t eventos/order-service:1.0.0 ./order-service
```

6. Componentes Docker

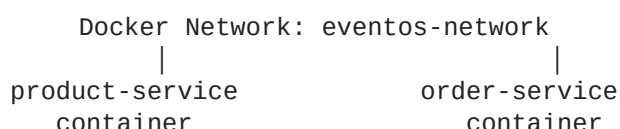
Componente	Função no projeto
Dockerfile	Receita declarativa da imagem. Cada serviço possui seu próprio Dockerfile.
Image	Artefato imutável com aplicação e dependências, como <code>eventos/product-service:1.0.0</code> .
Container	Instância em execução da imagem, com estado próprio durante sua execução.
Docker Engine	Serviço responsável por construir imagens, criar redes e executar containers.
Port Mapping	Publica portas para o host. Ex.: <code>8081:8081</code> para o catálogo.
Network	Rede virtual que permite que os containers se encontrem por nome.

Imagem e container não são a mesma coisa. A imagem é como um molde pronto; o container é esse molde rodando de verdade. Por isso, posso criar mais de um container usando exatamente a mesma imagem.

7 e 8. Comunicação e Docker Network

A rede bridge `eventos-network` conecta os dois containers. O serviço de pedidos recebe a URL `http://product-service:8081`. O próprio Docker resolve o nome `product-service` dentro da rede. O endereço `localhost` não é usado, porque, dentro do container de pedidos, ele apontaria para o próprio container e não para o catálogo.

```
docker network create eventos-network
docker run -d --name product-service --network eventos-network -p 8081:8081 eventos/product-service:1.0.0
docker run -d --name order-service --network eventos-network -e
PRODUCT_SERVICE_URL=http://product-service:8081 -p 8082:8082 eventos/order-service:1.0.0
```



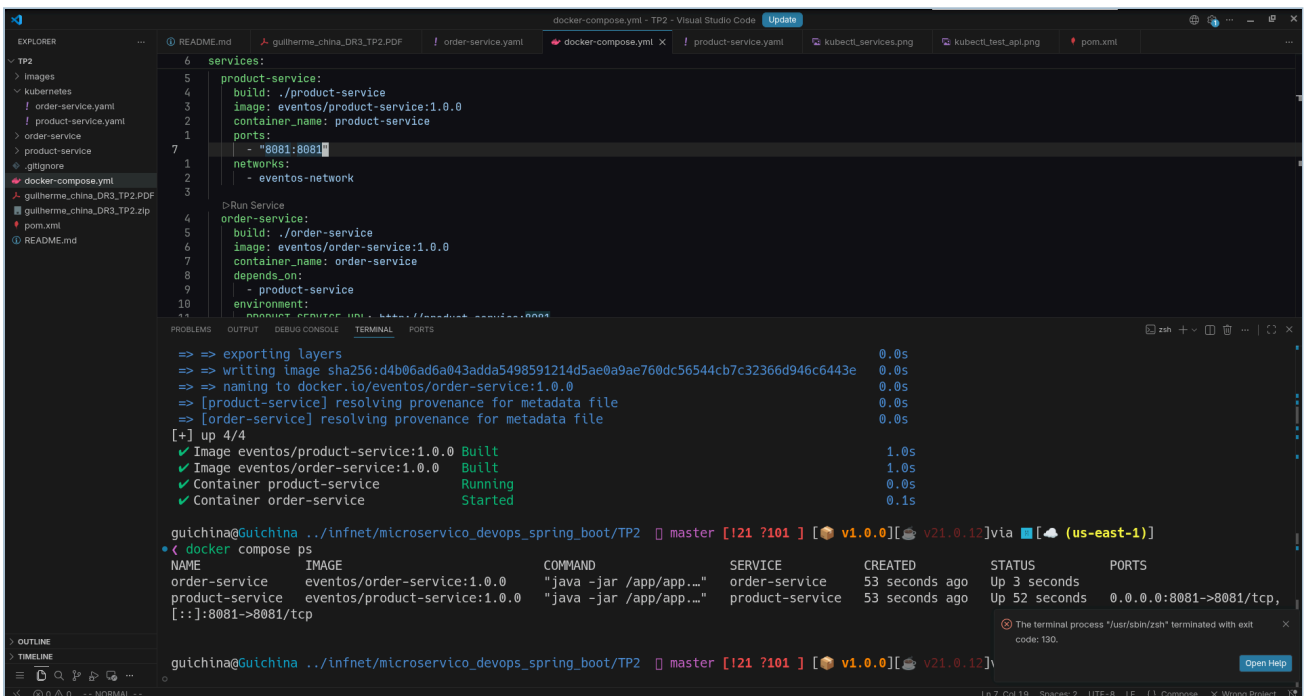
Docker Compose e migração

9. Docker Compose

Depois dos testes com comandos manuais, toda a configuração fica no `docker-compose.yml`. Nele estão os dois serviços, as portas 8081 e 8082, a rede `eventos-network` e a variável usada na comunicação. Com isso, todo o ambiente sobe com um único comando:

```
docker compose up --build -d
docker compose ps
curl http://localhost:8081/products
curl -X POST http://localhost:8082/orders -H 'Content-Type: application/json' -d
'{"productId":1,"quantity":2}'
```

No teste, os dois containers ficaram em execução e a criação do pedido retornou `{"id":1,"productId":1,"quantity":2,"status":"CRIADO"}`. Como a reserva só é criada depois da consulta ao catálogo, esse resultado confirma que a comunicação pela rede Docker funcionou.



The screenshot shows the Visual Studio Code interface with the `docker-compose.yml` file open. The file defines two services: `product-service` and `order-service`. The `product-service` is built from `./product-service` and uses the `eventos/product-service:1.0.0` image. The `order-service` is built from `./order-service` and uses the `eventos/order-service:1.0.0` image. Both services are connected to the `eventos-network`. The `order-service` depends on the `product-service`. The terminal output shows the build process for both services, including the export of layers, writing of image metadata, and the successful building of the images. The containers are then started, and the terminal shows the output of the `docker compose ps` command, indicating that both services are running.

```
services:
  product-service:
    build: ./product-service
    image: eventos/product-service:1.0.0
    container_name: product-service
    ports:
      - "8081:8081"
    networks:
      - eventos-network
  order-service:
    build: ./order-service
    image: eventos/order-service:1.0.0
    container_name: order-service
    depends_on:
      - product-service
    environment:
      PRODUCT_SERVICE_URL: http://product-service:8081

networks:
  eventos-network
```

```
> => exporting layers
=> writing image sha256:d4b06ad6a043adda5498591214d5ae0a9ae760dc56544cb7c32366d946c6443e
=> naming to docker.io/eventos/order-service:1.0.0
=> [product-service] resolving provenance for metadata file
=> [order-service] resolving provenance for metadata file
[+] up 4/4
✔ Image eventos/product-service:1.0.0 Built 1.0s
✔ Image eventos/order-service:1.0.0 Built 1.0s
✔ Container product-service Running 0.0s
✔ Container order-service Started 0.1s

guichina@Guichina: ~/infnet/microservico_devops_spring_boot/TP2 [master] [!21 ?101] [v1.0.0] [v21.0.12] via [us-east-1]
$ docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED     STATUS     PORTS
order-service       eventos/order-service:1.0.0         "java -jar /app/app..."           order-service  53 seconds ago  Up 3 seconds
product-service     eventos/product-service:1.0.0       "java -jar /app/app..."           product-service  53 seconds ago  Up 52 seconds  0.0.0.0:8081->8081/tcp,
[::]:8081->8081/tcp
```

Figura 1 — Build das imagens e containers `product-service` e `order-service` em execução pelo Docker Compose.

10. Equivalência entre Docker e Kubernetes

Docker	Kubernetes	Uso no projeto
Container	Pod	Cada réplica executa um Pod contendo o container Spring Boot.
Network	Rede do cluster + Service	O Service fornece DNS estável para os Pods.
Port Mapping	Service / port-forward	Os Services expõem as portas internas; port - forward permite acesso local.
Serviço da aplicação	Service	Os nomes dos serviços são usados para descoberta.
Múltiplos containers / Compose	Deployment	O Deployment declara imagem, réplicas e ambiente desejados.

Não foi preciso reescrever a aplicação para usar Kubernetes, porque o mesmo JAR continua rodando dentro da mesma imagem. O que muda é a forma de organizar a infraestrutura: o Deployment cuida das réplicas e o Service entrega um nome fixo para os outros serviços encontrarem a aplicação. Como a URL do catálogo já vem pela variável `PRODUCT_SERVICE_URL`, a adaptação ficou pequena.

Kubernetes, descoberta e balanceamento

11. product-service no Kubernetes

O arquivo `kubernetes/product-service.yaml` contém um Deployment e um Service. O Deployment informa a imagem, a porta 8081 e as verificações de prontidão e vida. Já o Service do tipo ClusterIP encontra os Pods pelo rótulo `app: product-service`.

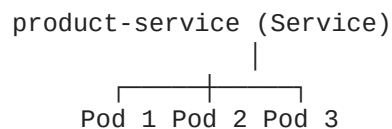
12. order-service no Kubernetes

O arquivo `kubernetes/order-service.yaml` segue a mesma ideia: tem Deployment e Service. Nele, a variável `PRODUCT_SERVICE_URL` recebe `http://product-service:8081`. Assim, o Kubernetes resolve o nome do Service pelo DNS e não é necessário colocar nenhum IP fixo no código.

```
kubectl apply -f kubernetes/product-service.yaml
kubectl apply -f kubernetes/order-service.yaml
kubectl get deployments,pods,services
kubectl port-forward service/order-service 8082:8082
```

13. Descoberta de serviço e balanceamento

O Deployment do `product-service` foi configurado com **três réplicas**. O Service encontra todos os Pods que têm o mesmo rótulo e manda as chamadas para os que estão disponíveis.



- a) O Service é o ponto de acesso estável para um conjunto de Pods e fornece descoberta por DNS.
- b) Vários Pods aumentam capacidade de atendimento e disponibilidade do microserviço.
- c) Se um Pod falhar, ele deixa de receber tráfego; o Deployment tenta recriá-lo e o Service continua usando os Pods saudáveis.
- d) O Service distribui as requisições entre os endpoints prontos, sem que o consumidor conheça os IPs individuais.

```
kubectl get pods -l app=product-service -o wide
kubectl get endpoints product-service
kubectl describe service product-service
```

Esses comandos demonstram os três Pods e os endpoints selecionados pelo Service em um cluster Kubernetes ativo.

```
5 package br.edu.infnet.order;
6
7 public class br.edu
8 {
9     public class Source: order-service
10     {
11         public OrderService(Long id) {
12             super("Pedido " + id + " não encontrado");
13         }
14     }
15 }
```

```
> kubectl get deployments,pods,services
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/mongo-deployment    0/1      1              1            100d
deployment.apps/order-service        1/1      1              1            49s
deployment.apps/product-service      3/3      3              3            49s
deployment.apps/webapp-deployment    1/1      1              1            100d

NAME                                READY    STATUS              RESTARTS      AGE
pod/mongo-deployment-58854cc66b-m7h2s 0/1      CrashLoopBackOff    12 (4m13s ago) 100d
pod/order-service-574d68cdb4-8qm65     1/1      Running              0              49s
pod/product-service-6f7b4cbbc-mm67z     1/1      Running              0              49s
pod/product-service-6f7b4cbbc-mqfpz     1/1      Running              0              49s
pod/product-service-6f7b4cbbc-nwxqp     1/1      Running              0              49s
pod/webapp-deployment-5766fd95c7-clbwt   1/1      Running              3 (28m ago)    100d

NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
service/kubernetes                   ClusterIP    10.96.0.1      <none>          443/TCP        149d
service/mongo-service                ClusterIP    10.96.229.137 <none>          27017/TCP      100d
service/order-service                 ClusterIP    10.103.58.142 <none>          8082/TCP        49s
service/product-service               ClusterIP    10.109.130.123 <none>          8081/TCP        49s
```

Figura 2 — Consulta dos Deployments, Pods e Services após a aplicação dos manifests Kubernetes.

```
11 package br.edu.infnet.order;
12
13 import org.springframework.http.HttpStatus;
14 import org.springframework.web.bind.annotation.*;
15 import java.util.List;
16
17 @RestController
18 @RequestMapping("/orders")
19 public class OrderController {
20     private final OrderService orderService;
21
22     public OrderController(OrderService orderService) {
23         this.orderService = orderService;
24     }
25
26     @GetMapping
27     public List<Order> findAll() {
28         return orderService.findAll();
29     }
30 }
```

```
guichina@Guichina: ~/infnet/microservico_devops_spring_boot/TP2 [master [121 7102] [v1.0.0] [v21.0.12] via [us-east-1]]
> curl -X POST http://localhost:8082/orders \
  -H 'Content-Type: application/json' \
  -d '{"productId":1,"quantity":2}'
{"id":2,"productId":1,"quantity":2,"status":"CRIADO"}

guichina@Guichina: ~/infnet/microservico_devops_spring_boot/TP2 [master [121 7102] [v1.0.0] [v21.0.12] via [us-east-1]]
>
```

Figura 3 — Comando utilizado para testar a criação de pedido pela API após a publicação no Kubernetes.

14. Avaliação e reflexão

A parte mais tranquila foi criar os endpoints REST em memória, porque cada serviço ficou pequeno e com uma responsabilidade bem definida. O ponto que mais exigiu atenção foi a comunicação entre eles: o endereço usado localmente não é igual ao usado no Docker nem no Kubernetes. A variável `PRODUCT_SERVICE_URL` resolveu isso sem precisar mexer na regra de negócio.

O principal aprendizado foi perceber a diferença prática entre Docker e Kubernetes. Docker deixou a execução dos serviços simples e repetível; Kubernetes acrescentou descoberta de serviços, controle de réplicas e distribuição de carga. A autoavaliação é **9/10**, pois os requisitos de microsserviços, Docker, Compose, Deployments, Services e réplicas foram implementados. O projeto ficou sem banco de dados de propósito, para manter o foco na parte de infraestrutura pedida no trabalho.